

1. **Express and the app:** Why a framework: Express on top of Node · The app object and app.listen · app.js builds, server.js starts
2. **Routes, req and res:** Routes: method + path + handler · Route parameters: req.params · Query strings: req.query · Reading the request: req · Answering: res · Exactly one response per request · express.Router(): routes under a prefix · Order of routes: the first match wins
3. **Middleware:** Middleware: (req, res, next) · Order is configuration · express.json() and req.body · Middleware you will use: static, cors, logging · Guards: middleware that can stop a request
4. **Controllers and the lifecycle:** Controllers: thin routes, testable logic · The life of a request
5. **Errors and 404s:** The error handler: four parameters, registered last · Raising errors: throw and next(err) · Async handlers: Express 4 versus Express 5 · The 404 catch-all

Express and the app

Why a framework: Express on top of Node

Express is a small library for Node.js that turns "one function for every request" into a list of **routes (method + path → function) and a **middleware pipeline** (small functions every request passes through), with helpers to read requests and send responses.**

```
// Node's http module alone: you do all the work
const http = require('http');
http.createServer((req, res) => {
  if (req.method === 'GET' && req.url === '/health') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ status: 'ok' }));
  } else {
    res.writeHead(404);
    res.end('Not found');
  }
}).listen(3000);

// The same server with Express
const express = require('express');
const app = express();
app.get('/health', (req, res) => res.json({ status: 'ok' }));
app.listen(3000);
```

Example. Add GET /tasks/7 to both versions. With http you write a regular expression for /tasks/<number>, pull the number out, and remember that /tasks/7?x=1 has a query string in req.url. With Express you add app.get('/tasks/:id', ...) and read req.params.id; the query string is already parsed into req.query, and every unknown URL still gets a 404 from Express without one more else.

Common mistake. Thinking Express is a different server or language that replaces Node. It is an npm package: express() returns an ordinary JavaScript function that Node's http module calls for each request. Every error you see is a Node error, and everything from Server-side JavaScript (modules, process.env, async/await) still applies.

The app object and app.listen

const app = express() **creates the app: an object that stores your middleware and routes in order.**
app.listen(port, callback) **starts a Node HTTP server that hands every incoming request to the app.**

```
const express = require('express');
const app = express(); // an empty pipeline

app.use(express.json()); // station 1
app.get('/health', (req, res) => { // station 2
  res.json({ status: 'ok' });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
```

```
console.log(`Listening on http://localhost:${PORT}`); // runs once
});
```

Example. Run the file with `node --watch src/server.js`. The terminal prints `Listening on http://localhost:3000` once and stays busy. In a second terminal, `curl http://localhost:3000/health` prints `{"status":"ok"}`. Save a change and `--watch` restarts the process, so the message appears again.

Common mistake. Starting the server twice (two terminals, or a forgotten one in the background) and getting `Error: listen EADDRINUSE: address already in use :::3000`. Only one program can own a port. Stop the old one (Ctrl+C in its terminal) or start this one on another port: `PORT=3001 node src/server.js`.

app.js builds, server.js starts

A well-organised Express project splits the program in two: `src/app.js` **builds the Express app and exports it with** `module.exports = app`; `src/server.js` **requires it and calls** `app.listen()`. **Building and starting are separate jobs, so tests can use the app without opening a port.**

```
// src/app.js – builds and exports; never listens
const express = require('express');
const logger = require('./middleware/logger');
const tasksRouter = require('./routes/tasks');

const app = express();
app.use(express.json());
app.use(logger);
app.get('/health', (req, res) => res.json({ status: 'ok' }));
app.use('/tasks', tasksRouter);
// ... 404 catch-all and error handler last
module.exports = app;

// src/server.js – one job: start listening
const app = require('./app');
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Tasks API listening on http://localhost:${PORT}`));
```

Example. An automated test does `const request = require('supertest');` `const app = require('./src/app');` and then `await request(app).get('/tasks')`. `supertest` gives the app a temporary in-memory connection, so twenty tests can run in a row without port 3000 ever being opened, and without a server left running afterwards.

Common mistake. Forgetting `module.exports = app` at the end of `app.js`. Then `require('./app')` returns an empty object `{}`, and `server.js` crashes with `TypeError: app.listen is not a function`. The opposite mistake, calling `listen` inside `app.js`, makes every test open a real port.

Routes, req and res

Routes: method + path + handler

A **route** connects one HTTP method and one path pattern to a **handler function**: `app.get('/tasks', listTasks)` means "when a GET request for `/tasks` arrives, call `listTasks(req, res)`".

```
app.get('/tasks', listTasks);           // GET    /tasks
app.post('/tasks', createTask);        // POST   /tasks
app.get('/tasks/:id', getTask);        // GET    /tasks/7, /tasks/abc ...
app.patch('/tasks/:id', updateTask);   // PATCH  /tasks/7
app.delete('/tasks/:id', deleteTask);  // DELETE /tasks/7
```

Example. With the five routes above: `GET /tasks` → `listTasks`; `GET /tasks/` → `listTasks` too (case and trailing slash ignored); `PUT /tasks/7` → no route has `PUT`, so Express answers 404 `Cannot PUT /tasks/7`; `GET /tasks/7/comments` → 404, because `/tasks/:id` has two segments and the URL has three.

Common mistake. Testing a POST route by typing its URL in the browser address bar. The browser always sends GET, so you get Cannot GET /tasks and conclude the route is broken. Use curl -X POST ..., Postman or a fetch call with method: 'POST'.

Route parameters: req.params

A segment written :name in a route path is a **route parameter**: it matches any one segment of the URL and its value appears, as a **string**, in req.params.name.

```
app.get('/users/:userId/tasks/:taskId', (req, res) => {
  // GET /users/7/tasks/42
  console.log(req.params);           // { userId: '7', taskId: '42' }
  const taskId = Number(req.params.taskId);
  if (Number.isNaN(taskId)) {
    return res.status(400).json({ error: 'taskId must be a number' });
  }
  res.json({ userId: req.params.userId, taskId });
});
```

Example. An echo route GET /echo/:msg answers { "echo": req.params.msg }. curl http://localhost:3000/echo/hello gives {"echo":"hello"}; /echo/hello%20world gives {"echo":"hello world"}; /echo/hello/world gives 404, because :msg takes exactly one segment.

Common mistake. Comparing the raw parameter with a number: tasks.find((t) => t.id === req.params.id). 7 === '7' is false (strict equality compares types too), so the task is never found and every request answers 404. Convert first: t.id === Number(req.params.id).

Query strings: req.query

The part of the URL after ? holds optional **key=value pairs**; Express parses it into the object req.query, whose values are **strings** (or arrays of strings when a key repeats). It never takes part in route matching.

```
// What Express does with "?done=false&page=2&tag=css&tag=html&q=hello+world"
const params = new URLSearchParams('done=false&page=2&tag=css&tag=html&q=hello+world');

console.log(params.get('page'), typeof params.get('page')); // a string
console.log(params.getAll('tag'));                          // repeated key
console.log(params.get('q'));                               // + is a space
console.log(params.get('sort'));                            // missing → null (undefined in req.query)

const done = params.get('done');
if (done) console.log('"false" is a non-empty string, so this runs!');
console.log('compare as text:', done === 'true');
```

Example. A tasks API filters on the server: GET /tasks?done=true. A correct controller writes const { done } = req.query; const list = done === undefined ? tasks : tasks.filter((t) => String(t.done) === done);. GET /tasks returns everything, ?done=true the finished tasks, ?done=false the others.

Common mistake. Putting the query string in the route: app.get('/tasks?done=true', ...). Express matches only the path, and ? has a special meaning in Express 4 patterns, so this route never matches what you expect. Register /tasks and read req.query.done inside.

Reading the request: req

req is the request as an object: the method and URL, the route parameters, the parsed query string and body, and the headers. Express fills most of it before your handler runs.

The parts of req you will use (request: POST /api/tasks/7?notify=true, router mounted at /api/tasks)

Property	Value in the example	Filled by
req.method	'POST'	Node
req.originalUrl	'/api/tasks/7?notify=true'	Express: the full URL as sent
req.baseUrl	'/api/tasks'	Express: the mount path of the current router

Property	Value in the example	Filled by
req.path	<code>'/7'</code>	Express: the path inside this router, without the query
req.params	<code>{ id: '7' }</code>	Express, from the route pattern <code>/:id</code>
req.query	<code>{ notify: 'true' }</code>	Express, from the query string
req.body	<code>{ title: 'Study' }</code>	<code>express.json()</code> (undefined without it)
req.get('Authorization')	<code>'Bearer eyJ...'</code>	Node (headers), read without caring about case
req.user	<code>{ id: 1, name: 'Ana' }</code>	Your own auth middleware, if any

Example. A logger that prints `${req.method} ${req.originalUrl}` shows `POST /api/tasks/7?notify=true` wherever it is registered. A simpler logger that uses `req.url` prints the same thing as long as the logger is registered with `app.use(logger)` at application level; registered inside a router, it would print only `/7?notify=true`.

Common mistake. Reading `req.headers['Content-Type']` and getting undefined. Node lower-cases every header name, so the key is `'content-type'`. Use `req.get('Content-Type')`, which works with any capitalisation.

Answering: res

res builds the response: `res.status(code)` **sets the status**, `res.set(name, value)` **a header**, and **one sending method ends it:** `res.json(data)`, `res.send(text)`, `res.sendStatus(code)` **or** `res.end()`.

The res methods you will use most

Call	Sends?	Result
<code>res.status(404)</code>	No	Sets the status code; returns res for chaining
<code>res.set('Location', '/tasks/8')</code>	No	Sets a header (also <code>res.location(url)</code>)
<code>res.json({ id: 8 })</code>	Yes	JSON body with Content-Type: application/json
<code>res.send('<h1>Hi</h1>')</code>	Yes	Text body, Content-Type: text/html by default
<code>res.sendStatus(204)</code>	Yes	Only the status (and its reason as text, except 204)
<code>res.status(204).end()</code>	Yes	No body at all

Example. A complete creation answer: `res.status(201).location(/tasks/${task.id}).json(task)`. On the wire: HTTP/1.1 201 Created, a Location: /tasks/4 header, Content-Type: application/json; charset=utf-8, and the body `{"id":4,"title":"Study","done":false}`.

Common mistake. Writing `res.status(201)` and stopping there, expecting it to answer. It only sets a number on the parcel: no sending method was called, so the request **hangs** until the client gives up. Every path through a handler must end with exactly one sending call.

Exactly one response per request

Every request must get one response: zero means the request hangs; two means Node throws `Error [ERR_HTTP_HEADERS_SENT]: Cannot set headers after they are sent to the client`. **Write** `return res....` **whenever more code follows**.

```
// Bug: without return, a body without title sends TWO responses
function createTask(req, res) {
  const { title } = req.body;
  if (!title) res.status(400).json({ error: 'title is required' });
  const task = { id: tasks.length + 1, title, done: false }; // still runs!
  tasks.push(task);
  res.status(201).json(task); // → Error [ERR_HTTP_HEADERS_SENT]
}

// Fixed: return stops the function after the 400
function createTask(req, res) {
  const { title } = req.body;
  if (!title) return res.status(400).json({ error: 'title is required' });
  const task = { id: tasks.length + 1, title, done: false };
  tasks.push(task);
  res.status(201).json(task);
}
```

Example. With the buggy version, `POST /tasks` with `{}` returns `400` to the client, which looks right. But the function also pushed a task with `title: undefined` into the array, and the terminal shows `Error [ERR_HTTP_HEADERS_SENT]`. The next `GET /tasks` lists a task with no title: the bug corrupted data while the client saw a correct status.

Common mistake. Fixing the error message by wrapping the second `res.json` in `try/catch`. The real bug is that the code kept running after it answered; the fix is `return` (or `if/else`), so only one branch ever sends.

express.Router(): routes under a prefix

`express.Router()` creates a mini-app that holds routes (and middleware) for one resource. `app.use('/tasks', tasksRouter)` mounts it: requests whose path starts with `/tasks` enter the router, and inside it the prefix is removed, so `router.get('/')` means `GET /tasks`.

```
// src/routes/tasks.js – thin: paths → controller functions
const express = require('express');
const { listTasks, createTask, getTask } = require('../controllers/tasksController');

const router = express.Router();
router.get('/', listTasks);      // GET /tasks
router.post('/', createTask);   // POST /tasks
router.get('/:id', getTask);    // GET /tasks/7
module.exports = router;

// src/app.js
app.use('/tasks', require('./routes/tasks'));
```

Example. Say you move the API under `/api`. Only one line changes: `app.use('/api/tasks', tasksRouter)`. `GET /api/tasks/7` now enters the router, which still sees `/7` and still calls `getTask`; nothing in `routes/tasks.js` or the controller changes.

Common mistake. Repeating the prefix inside the router: `router.get('/tasks', listTasks)` while `app.js` mounts it at `/tasks`. The real path becomes `/tasks/tasks`, and `GET /tasks` answers `404`. Inside a router, paths are relative to the mount path.

Order of routes: the first match wins

Express tries routes in the order they were registered and runs the first one whose method and path match. It never looks for the "most specific" route, so a pattern like `/tasks/:id` placed before `/tasks/stats` captures `stats` as an id.

```
// Wrong order: GET /tasks/stats → getTask with id 'stats' → 404
router.get('/:id', getTask);
router.get('/stats', taskStats); // never reached

// Right order
router.get('/stats', taskStats); // fixed path first
router.get('/:id', getTask);
```

Example. A tasks API needs a `GET /tasks/stats` endpoint. A developer adds it at the bottom of `routes/tasks.js`, under `/:id`, and gets `{"error": "Task not found"}`. Nothing is wrong with the stats code: moving the line above `router.get('/:id', ...)` fixes it.

Common mistake. "Express picks the best-matching route." It does not compare routes at all; it walks the list in order. The same goes for middleware: order in the file is the configuration.

Middleware

Middleware: (req, res, next)

A **middleware** is a function (`req, res, next`) that Express runs for each request in registration order. It may change `req` or `res`, and then it must do one of two things: call `next()` to pass the request on, or send a response

to end it. Doing neither leaves the request hanging.

```
// Logs, then passes the request on
function logger(req, res, next) {
  console.log(`${req.method} ${req.url}`);
  next(); // forget this and every request hangs
}

// Answers early, or passes on
function requireJson(req, res, next) {
  if (req.method === 'POST' && !req.is('application/json')) {
    return res.status(415).json({ error: 'Send JSON' }); // ends here
  }
  next();
}

app.use(logger);
app.use(requireJson);
```

Example. Request GET /tasks on a typical tasks app (json parser, logger, /health, /echo/:msg, the tasks router, 404, error handler): `express.json()` runs and calls `next()` (no body to read); logger prints GET /tasks and calls `next()`; GET /health and GET /echo/:msg do not match and are skipped; the router mounted at /tasks matches, its GET / route runs `listTasks`, which answers 200. The layers after it (the 404 catch-all, the error handler) never run.

Common mistake. Writing a logger without `next()`. The terminal prints GET /health (so "the middleware works"), but the browser spins forever and `curl` never returns. Remember the rule: forget `next()` and every request hangs forever.

Order is configuration

Express runs layers in the order they were registered, so the order of `app.use` and route lines decides what each request meets: parsers and loggers first, guards before what they protect, routes, then the 404 catch-all, and the error handler last.

- 1. **Global middleware:** `cors()`, `express.json()`, the logger, `express.static(...)`.
- 2. **Public routes:** GET /health, the login route.
- 3. **Guards:** `app.use('/api', requireAuth)` (everything below it under /api needs a token).
- 4. **Protected routes and routers:** `app.use('/api/tasks', tasksRouter)`.
- 5. **404 catch-all:** `app.use((req, res) => res.status(404).json(...))`.
- 6. **Error handler**, with four parameters, always last.

Example. In the "Protected API" preset of the pipeline simulator, GET /api/health is registered **above** `app.use('/api', requireAuth)`, so it stays public: health checks work without a token. GET /api/me is registered below the guard, so without Authorization: Bearer demo-token it answers 401.

Common mistake. Moving `app.use(express.json())` to the bottom of `app.js` "to keep the setup tidy". Every route above it now sees `req.body` as undefined, and POST /tasks crashes with Cannot destructure property 'title' of 'req.body'.

express.json() and req.body

`express.json()` is built-in middleware that reads the request body and, when the request has Content-Type: application/json, parses it into `req.body`. Without it `req.body` is undefined; with the wrong Content-Type it is `{}` (Express 4); with broken JSON it sends an error with status 400 to your error handler.

JavaScript + shell

```
app.use(express.json()); // before the routes!

app.post('/tasks', (req, res) => {
  console.log(req.body); // { title: 'Buy milk' }
  res.status(201).json({ received: req.body });
});

// The client must say the body is JSON:
```

```
// curl -X POST http://localhost:3000/tasks \
// -H "Content-Type: application/json" \
// -d '{"title":"Buy milk"}'
```

Example. Run a `curl POST /tasks` command without the `-H "Content-Type: application/json"` line. `curl` then labels the body `application/x-www-form-urlencoded`, `express.json()` ignores it, and `createTask` receives `req.body = {}`: a controller that echoes the body answers `{"received":{}}`, and a validating controller answers `400 title is required`, although the JSON was right there.

Common mistake. Blaming the controller when `req.body` is undefined. That value means no body parser ran before the route: `express.json()` is missing or registered after it. `{}` means the parser ran but skipped the body: check the client's `Content-Type`.

Middleware you will use: static, cors, logging

A handful of middleware covers most projects: `express.json()` and `express.urlencoded()` (**bodies**), `express.static('public')` (**files for the browser**), `cors()` (**calls from other origins**) and a **request logger** (**morgan or your own**). The first three come with Express; `cors` and `morgan` are npm packages you install.

Common middleware

Middleware	Comes from	What it does	Register
<code>express.json()</code>	Express	JSON body → <code>req.body</code>	Early, before routes
<code>express.urlencoded({ extended: false })</code>	Express	HTML form body → <code>req.body</code>	Early, before routes
<code>express.static('public')</code>	Express	Sends files from a folder, else <code>next()</code>	Before the API routes
<code>cors()</code>	<code>npm install cors</code>	Adds CORS headers for browser calls from other origins	First, before routes
<code>morgan('dev')</code>	<code>npm install morgan</code>	One log line per request (method, URL, status, time)	Early
<code>logger (yours)</code>	Your file	Prints METHOD URL, then <code>next()</code>	Early

Example. An `app.js` that serves a front end and an API starts with `app.use(cors()); app.use(express.json()); app.use(logger); app.use(express.static('public'))`; A browser asking for `/` gets `public/index.html` from `express.static`; the page's script calls `fetch('/api/tasks')`, which finds no file, falls through `express.static`, and reaches the `tasks` router.

Common mistake. Error: Cannot find module 'cors': only `express.json`, `express.urlencoded` and `express.static` ship with Express. `cors` and `morgan` must be installed first (`npm install cors morgan`), which also adds them to dependencies in `package.json`.

Guards: middleware that can stop a request

A guard is middleware that checks something (usually who is calling) and either stops the request with an error response (401, 403) or adds what it learned to `req` (such as `req.user`) and calls `next()`.

```
function requireAuth(req, res, next) {
  const header = req.get('Authorization');           // "Bearer <token>"
  if (header !== 'Bearer demo-token') {
    return res.status(401).json({ error: 'Log in first' });
  }
  req.user = { id: 1, name: 'Ana' };                // a real app verifies a JWT here
  next();
}

app.get('/api/health', health);                      // public: above the guard
app.use('/api', requireAuth);                        // guards everything below under /api
app.get('/api/me', (req, res) => res.json({ user: req.user }));
```


Example. `curl http://localhost:3000/api/me → 401 {"error":"Log in first"}`. `curl -H "Authorization: Bearer demo-token" http://localhost:3000/api/me → 200 {"user":{"id":1,"name":"Ana"}}`. `curl http://localhost:3000/api/health → 200` without a token, because that route is registered before the guard.

Common mistake. Registering the guard after the route it should protect. The route answers first, `req.user` is undefined, and `res.json({ user: undefined })` sends `{}` with status 200: no error anywhere, just an unprotected endpoint.

Controllers and the lifecycle

Controllers: thin routes, testable logic

A controller is the plain function that does a route's work (read the request, validate, fetch or change data, answer). Controllers live in `controllers/`, the route file only maps paths to them, so each part is short and the logic can be tested on its own.

```
// controllers/tasksController.js (no Express needed here)
const tasks = [{ id: 1, title: 'Write the API skeleton', done: true }];

function getTask(req, res) {
  const task = tasks.find((t) => t.id === Number(req.params.id));
  if (!task) return res.status(404).json({ error: 'Task not found' });
  res.json(task);
}

// A fake res that remembers what the controller did
function fakeRes() {
  const res = { statusCode: 200, body: undefined };
  res.status = (code) => { res.statusCode = code; return res; };
  res.json = (data) => { res.body = data; return res; };
  return res;
}

const r1 = fakeRes();
getTask({ params: { id: '1' } }, r1);
console.log(r1.statusCode, JSON.stringify(r1.body));

const r2 = fakeRes();
getTask({ params: { id: '99' } }, r2);
console.log(r2.statusCode, JSON.stringify(r2.body));
```

Example. A small `routes/tasks.js` has three lines of real code: create a router, `router.get('/', listTasks)`, `router.post('/', createTask)`. When the project gets a database, the array inside `tasksController.js` is replaced by calls to a database model; the route file and `app.js` do not change at all.

Common mistake. Writing the logic inline in the route file "because it is only a few lines": `router.post('/', (req, res) => { ...30 lines... })`. The file grows unreadable and the logic cannot be tested on its own: keep handlers in `controllers/`. And a controller should never require the app or call `app.listen`, so that it stays a plain, testable function.

The life of a request

A request travels through one pipeline: Node receives it and hands it to the app; each middleware in order passes it on; the router picks the route; the controller answers. Any `next(err)` or thrown error jumps to the error handler; a request nobody answers reaches Express's own 404.

- 1. Node parses the request line and headers and calls the app.
- 2. Express adds its helpers (`req.query`, `req.get`, `res.json...`) and the X-Powered-By: Express header.
- 3. Global middleware, in order: `cors()`, `express.json()` (fills `req.body`), the logger.
- 4. Guards on a prefix: `app.use('/api', requireAuth)` (fills `req.user` or answers 401).
- 5. Routes and routers: the first method + path match runs its controller, which sends the response.
- 6. No match: your 404 catch-all, or Express's Cannot GET /x.

- 7. On `next(err)` or a thrown error: everything else is skipped until the 4-parameter error handler.

Example. `POST /tasks` with a broken JSON body: `express.json()` fails to parse it and calls `next(err)` with status 400 (step 3); the logger, the routes and the 404 catch-all are all skipped, because an error is pending (step 7); the error handler answers 400 `{"error": "Expected property name or '}' in JSON at position 1 ..."}`. The controller never ran, and the logger never printed a line.

Common mistake. Assuming every middleware runs for every request. A layer only runs if the request **reaches** it: an earlier response ends the journey, an error makes Express skip ordinary layers, and `app.use('/api', ...)` never sees `/health`.

Errors and 404s

The error handler: four parameters, registered last

An **error-handling middleware** is a function with **four parameters**, (`err`, `req`, `res`, `next`). Express recognises it **only by that count, skips it during normal requests, and sends it every error passed with `next(err)` or thrown in a handler. Register one, after every route.**

```
// How Express tells the two kinds apart: it counts the parameters
const errorHandler = (err, req, res, next) => {};
const looksLikeOne = (err, req, res) => {};

console.log('4 parameters →', errorHandler.length, '(error handler)');
console.log('3 parameters →', looksLikeOne.length, '(ordinary middleware!)');

// Express 4, simplified (Layer.handle_error / handle_request):
function isErrorHandler(fn) { return fn.length === 4; }
console.log(isErrorHandler(errorHandler), isErrorHandler(looksLikeOne));
```

Example. A test route `GET /boom` calls `next(new Error('Boom!'))`. With `app.use((err, req, res, next) => { console.error(err.message); res.status(err.status || 500).json({ error: err.message }); })` at the bottom, `curl` prints `{"error": "Boom!"}` with status 500, the terminal prints `Boom!`, and the server keeps running.

Common mistake. Deleting the "unused" `next` parameter from the error handler to please a linter. With three parameters it is no longer an error handler: errors skip it, the client gets Express's HTML error page, and on normal requests that reach it Express calls it as (`req`, `res`, `next`), so your `res.status` is really `next.status` and it throws `TypeError: res.status is not a function`.

Raising errors: throw and `next(err)`

There are two ways to send a failure to the error handler: call `next(err)` (**works everywhere**), or **throw inside a handler while it is still running synchronously (Express catches it)**. From then on Express skips ordinary layers until a four-parameter error handler.

```
// Both reach the error handler
app.get('/boom', (req, res, next) => {
  next(new Error('Boom!')); // explicit
});
app.get('/boom2', (req, res) => {
  throw new Error('Boom!'); // caught by Express (synchronous)
});

// An error with a status, for the central handler
function getTask(req, res, next) {
  const task = tasks.find((t) => t.id === Number(req.params.id));
  if (!task) {
    const err = new Error('Task not found');
    err.status = 404;
    return next(err);
  }
  res.json(task);
}
```

Example. In a tasks app without `express.json()`, `POST /tasks` makes `createTask` run `const { title } = req.body` with `req.body` undefined. JavaScript throws `TypeError: Cannot destructure property 'title' of 'req.body' as it is undefined.`; Express catches it, skips the 404 catch-all, and the error handler answers 500 with that message. The server survives.

Common mistake. Calling `next('Task not found')` with a plain string. Express treats it as an error, but the handler's `err.message` and `err.status` are undefined, so the client gets 500 and `{}`. Always pass an `Error` object (with a status when it is not a 500).

Async handlers: Express 4 versus Express 5

In **Express 4**, an `async` handler whose promise rejects never reaches your error handler: on Node 15+ the unhandled rejection **crashes the server**. Catch it yourself with `try/catch` and `next(err)`. **Express 5** forwards rejected promises to the error handler automatically.

```
// Express 4: a rejection here crashes the process
app.get('/stats', async (req, res) => {
  const stats = await db.getStats();    // rejects: database offline
  res.json(stats);
});

// Works in Express 4 and 5
app.get('/stats', async (req, res, next) => {
  try {
    const stats = await db.getStats();
    res.json(stats);
  } catch (err) {
    next(err);                          // → error handler → 500 JSON
  }
});

// Optional helper once the pattern is familiar
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);
router.get('/', asyncHandler(listTasks));
```

Example. In the pipeline simulator, preset "Bug: async route without try/catch": GET `/stats` with Express 4 ends in a crash and `curl` prints Empty reply from server. Switch the version to Express 5 and the same code answers 500 `{"error": "Database is offline"}`; or stay on 4 and choose the `try/catch + next(err)` code.

Common mistake. Adding `try/catch` but only logging: `catch (err) { console.error(err); }`. The crash is gone, but now nothing answers the request, so it hangs. The catch must end the request too: `next(err)`, or a response of its own.

The 404 catch-all

When no layer answers, Express sends its own 404 HTML page Cannot GET /x. An API adds a **catch-all** middleware after every route, `app.use((req, res) => res.status(404).json({ ... })),` so unknown URLs get JSON too.

```
// ... every route and router above ...

app.use((req, res) => {                                // 404 catch-all
  res.status(404).json({ error: `Not found: ${req.method} ${req.originalUrl}` });
});

app.use((err, req, res, next) => {                     // error handler, last
  console.error(err.message);
  res.status(err.status || 500).json({ error: err.message });
});
```

Example. Without the catch-all, `curl -i http://localhost:3000/nope` shows HTTP/1.1 404 Not Found, Content-Type: text/html and a small HTML page with `<pre>Cannot GET /nope</pre>`. With it: 404, Content-Type: application/json and `{"error": "Not found: GET /nope"}`, which a fetch call can read with `response.json()`.

Common mistake. Registering the catch-all near the top of `app.js`, next to the other `app.use` lines. It matches every request, so every route below it is unreachable and the whole API answers 404. Its place is after the last route.